VulnLens

Smart Contract Security Analysis Report

Contract Information

Contract	access_control.sol
Solidity Version	^0.8.20
Source Lines	19
Generated	17 October 2026, 03:39

Executive Summary

VulnLens identified **3** finding(s) in the submitted Solidity contract. Findings are ranked from High to Medium to Low to Informational. Each finding contains static evidence, relevant source context, contextual reasoning, and remediation guidance.

High	Medium	Low	Informational
0	**0**	**1**	**2**

Ranked Security Findings

Finding 1 — Low

Detector	missing-zero-check
Affected Function	changeOwner
Detected Source Lines	31, 33
Static Confidence	Medium

Static Analysis Evidence

• owner = newOwner (access_control.sol#33)

Relevant Solidity Context

```
1: // SPDX-License-Identifier: MIT
2: pragma solidity ^0.8.20;
3:
4: contract AccessControlVulnerable {
5:     address public owner;
```

```
6:     uint256 public value;
7:
8:     constructor() {
9:         owner = msg.sender;
10:     }
11:
12:     function setValue(uint256 _value) external {
13:         value = _value;
14:     }
15:
16:     function changeOwner(address newOwner) external {
17:         owner = newOwner;
18:     }
19: }
```

Contextual LLM Assessment

The changeOwner function assigns newOwner directly to the owner variable without verifying that newOwner is not address(0). This allows any caller to set the owner to the zero address. However, the contract does not use the owner variable for any access control or critical operations, so the practical impact is low. The function also lacks any access control, which is a separate, more severe issue.

Exploitability: The function is external and has no access control, so any attacker can call changeOwner(address(0)) and set owner to zero. Because the owner variable is not referenced elsewhere, this does not result in loss of funds or contract compromise. The exploitability is therefore low.

Attack Scenario: An attacker calls changeOwner(address(0)). The owner variable becomes 0x0. No further actions are possible because the contract does not enforce any owner-only operations. The attacker gains no advantage.

Remediation Steps

1. Add a check `require(newOwner != address(0), "Invalid new owner");` to prevent setting the owner to the zero address.
2. Add an access control modifier (e.g., `onlyOwner`) to the changeOwner function to restrict who can change the owner.

Finding 2 — Informational

Detector	solc-version
Affected Function	Not available
Detected Source Lines	3
Static Confidence	High

Static Analysis Evidence

• VerbatimInvalidDeduplication

• FullInlinerNonExpressionSplitArgumentEvaluationOrder

• MissingSideEffectsOnSelectorAccess.

• ^0.8.20 (access_control.sol#3)

Relevant Solidity Context

```
1: // SPDX-License-Identifier: MIT
2: pragma solidity ^0.8.20;
3:
4: contract AccessControlVulnerable {
5:     address public owner;
6:     uint256 public value;
7:
8:     constructor() {
```

Contextual LLM Assessment

The contract is compiled with Solidity ^0.8.20, which contains known severe bugs (VerbatimInvalidDeduplication, FullInlinerNonExpressionSplitArgumentEvaluationOrder, MissingSideEffectsOnSelectorAccess). These compiler issues can corrupt generated bytecode, affecting contract semantics and potentially leading to unauthorized access or state manipulation.

Exploitability: Low to moderate; exploitation requires the specific buggy compiler version to be used during compilation, which is uncommon in typical deployment pipelines, but a malicious actor could deliberately compile the contract with the vulnerable compiler to trigger the bugs.

Attack Scenario: An attacker compiles the AccessControl contract with a Solidity ^0.8.20 compiler that triggers the FullInlinerNonExpressionSplitArgumentEvaluationOrder bug, causing incorrect selector access handling and possibly allowing unauthorized privilege escalation.

Remediation Steps

1. Upgrade the Solidity compiler to a version without the reported bugs (e.g., ^0.8.21 or later) and re■compile the contract.
2. Re■deploy the contract using the patched compiler and verify the bytecode for any anomalies before publishing.

Finding 3 — Informational

Detector	naming-convention
Affected Function	setValue
Detected Source Lines	23
Static Confidence	High

Static Analysis Evidence

Parameter AccessControlVulnerable.setValue(uint256)._value (access_control.sol#23) is not in mixedCase

Relevant Solidity Context

```
1: // SPDX-License-Identifier: MIT
2: pragma solidity ^0.8.20;
3:
4: contract AccessControlVulnerable {
5:     address public owner;
6:     uint256 public value;
7:
8:     constructor() {
9:         owner = msg.sender;
10:     }
11:
12:     function setValue(uint256 _value) external {
13:         value = _value;
14:     }
15:
16:     function changeOwner(address newOwner) external {
17:         owner = newOwner;
18:     }
19: }
```

Contextual LLM Assessment

The parameter `_value` in the `setValue` function (line 12) uses an underscore prefix and all lowercase letters, which does not conform to the mixedCase (camelCase) naming convention. The detector reports this at line 23, but the actual code is at line 12. This is an informational finding related to code style and does not affect the contract's functionality or security.

Exploitability: Not exploitable. This is a style issue with no impact on the contract's behavior or security.

Attack Scenario: No attack scenario exists; the naming convention does not introduce any vulnerability.

Remediation Steps

1. Rename the parameter from `_value` to a mixedCase name, e.g., `value` or `newValue`, to adhere to Solidity naming conventions.
2. Review and enforce consistent naming conventions across the entire contract to improve readability and maintainability.

Analysis Methodology

• The Solidity source file is validated before analysis.

• Slither performs static analysis on the submitted contract.

• Detector results are converted into structured security evidence.

• Relevant Solidity source context is selected around reported locations.

• High, medium, and low impact findings are supplied to the configured LLM for contextual reasoning.

• Findings are ranked from High to Medium to Low to Informational.

• The final report combines static-analysis evidence, source context, contextual reasoning, and remediation guidance.

Scope and Limitations

• VulnLens is a pre-deployment smart-contract analysis system.

• The system does not deploy contracts.

• The system does not interact with a blockchain or ledger.

• The system does not use wallets or private keys.

• Static-analysis findings identify suspicious or security-relevant patterns and require contextual review.

• LLM output is contextual reasoning over supplied evidence and source code and should not replace independent security review.

• The absence of a finding does not prove that a contract is completely secure.

Generated by VulnLens